

Experiment Report

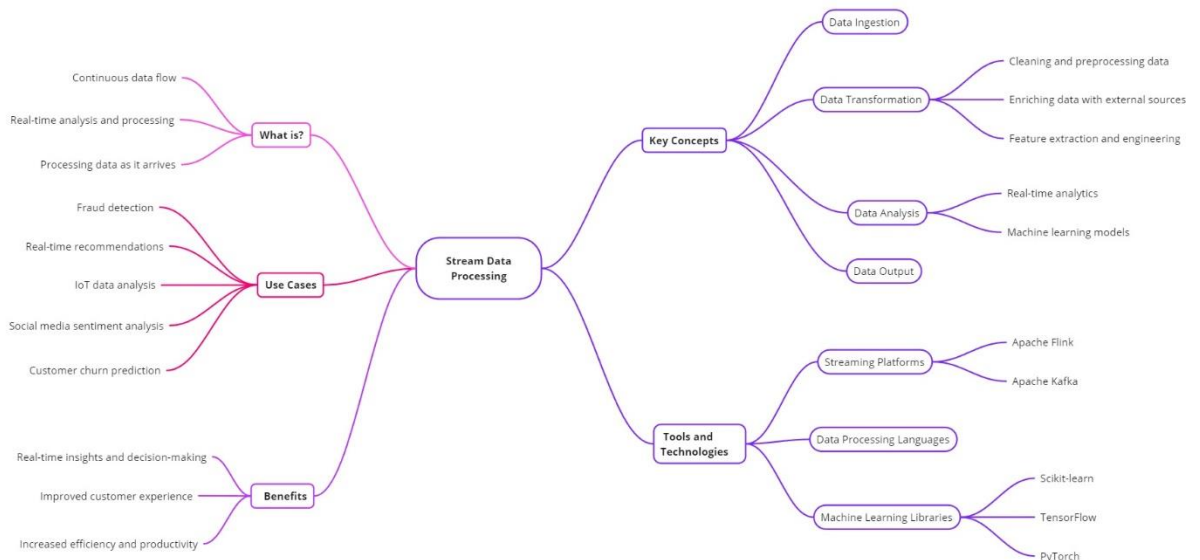
Student: Denisenko Sofiia 狄苏菲

Student id: 1820249051

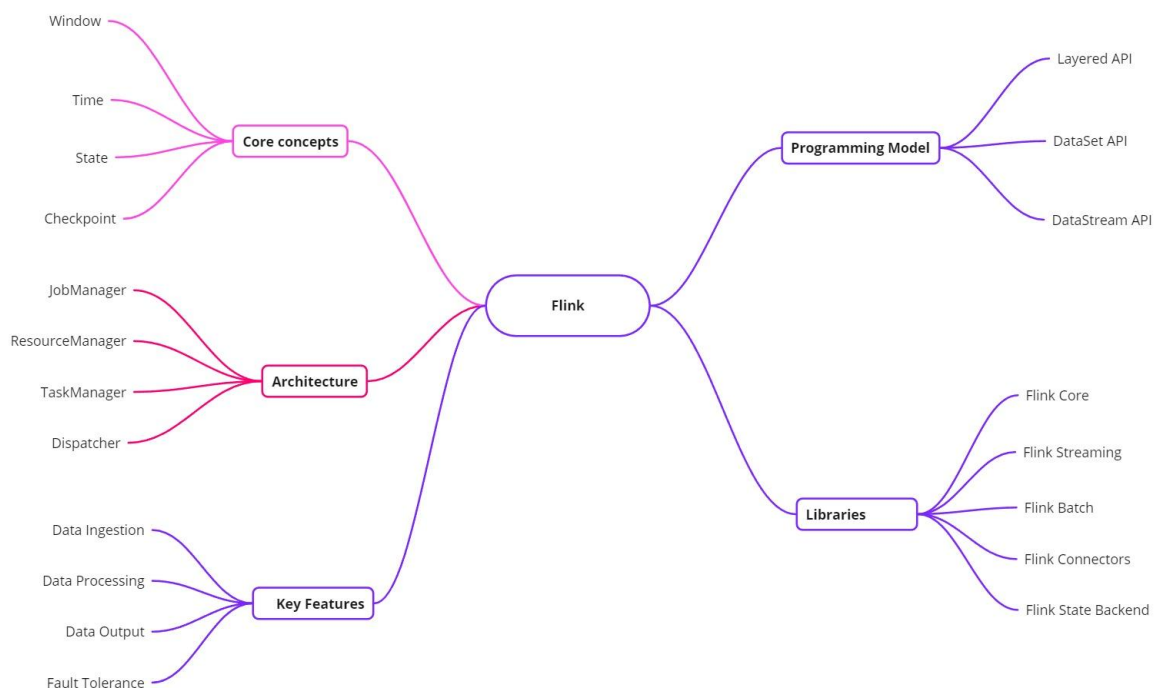
Course: Big Data Analysis Technology

Experiment Topic: Stream Processing and Flink

Mind Map Stream Data Processing



Mind Map Flink



Question 1

Explain and illustrate the concepts of State and Checkpoint, as well as their working principles, and how the fault recovery can be achieved through State and Checkpoint. (Using text and the diagram)

Answer:

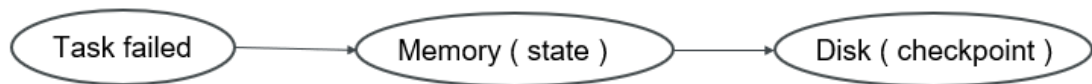
1. State:

Generally, refers to the state of a specific Task/Operator (the state of an operator indicates some intermediate results generated by some operators during operation). State data is stored in the Java heap memory/TaskManager node memory by default. State can be recorded, and data can be restored in the event of failure.

2. Checkpoint:

Represents a global state snapshot of a Flink job at a specific moment, it contains the state of all Tasks/Operators. It can be understood that Checkpoint is the timed and persistent storage of State data. Basically, Checkpoint is a snapshot of the entire Flink job's state, allowing it to recover if anything goes wrong.

Fault Recovery:



So, a Flink task encounters a problem and crashes. Before the failure, the task was diligently processing data and accumulating its state, storing it in memory. Flink already taken regular checkpoints of the job's state. The checkpoints are written to durable storage (like HDFS) to ensure they're safe even if a TaskManager crashes.

Question 2

With reference to the Flink architecture diagram, explain the components and working principles of two architectures in Flink's **YARN mode**: the task submission architecture and the runtime architecture. (Flink runtime architectures include Local mode, Standalone cluster mode, and **YARN mode**).

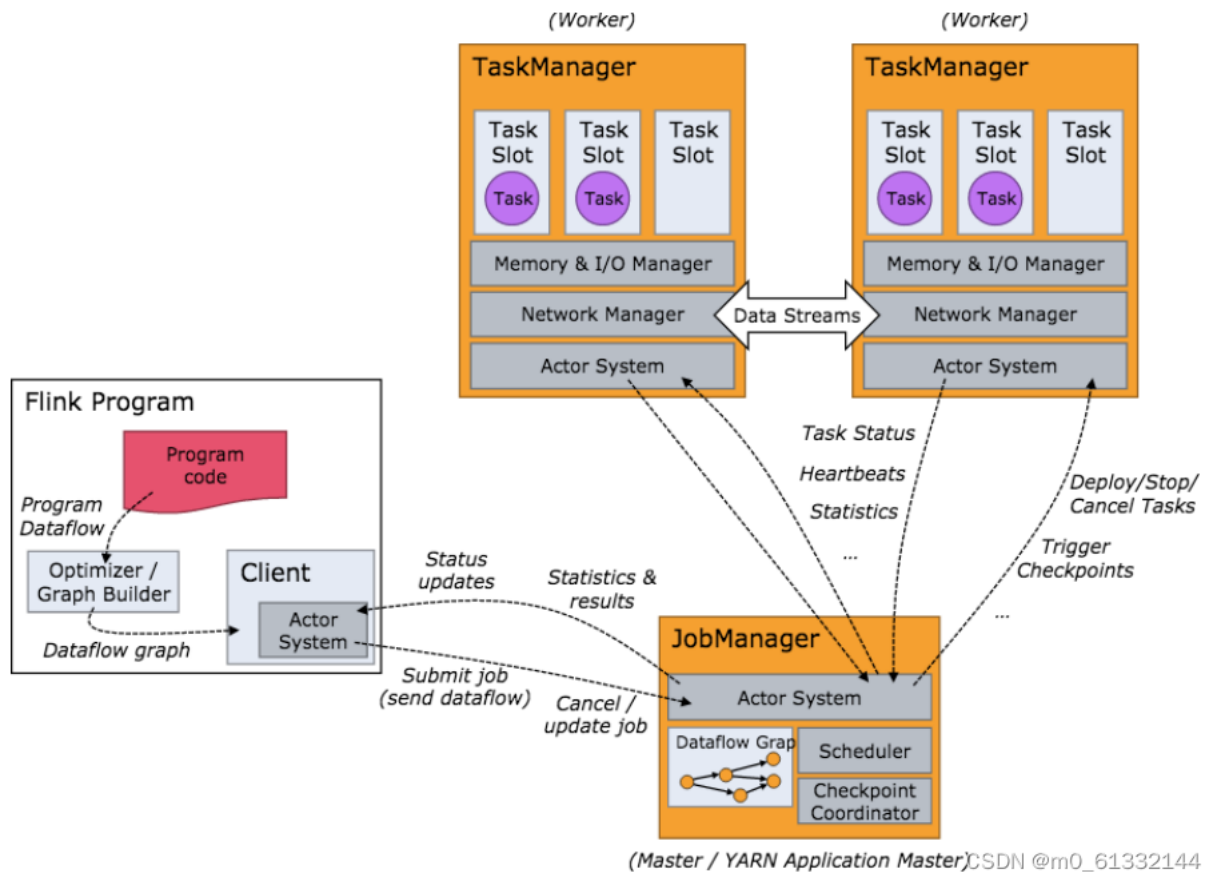
Answer:

YARN (Yet Another Resource Negotiator): YARN is a resource management system in Hadoop. It provides a framework for managing cluster resources and running applications. Flink can leverage YARN for launching and managing its applications in a distributed environment.

YARN Mode components:

- **Client:** The client first submits an application request to the YARN ResourceManager.
- **YARN ResourceManager:** YARN is responsible for managing resources and starting the Application Master (which acts as Flink's JobManager here).
- **Application Master:** Serves as Flink's JobManager in YARN, responsible for task scheduling and management.
- **TaskManager:** NodeManager in YARN starts the TaskManagers, which execute tasks and return the results.

In this mode, Flink acquires resources dynamically through YARN, making it suitable for large-scale cluster deployments.



Task Submission Architecture:

Focus:

This architecture handles the initial submission of the Flink job to the YARN cluster and the allocation of resources for the job.

Process:

1. **Submission:** The Flink Client sends the packaged Flink job to the YARN ResourceManager.
2. **Resource Request:** The client requests containers (resources) from the ResourceManager for the job.

3. Allocation: The ResourceManager allocates containers to specific NodeManagers based on availability.
4. Launch: The NodeManagers launch the Flink job (within its containers) on their respective nodes.

Runtime Architecture:

Focus:

This architecture handles the execution and management of the Flink job once it has been launched on the YARN cluster.

Process:

1. Job Initialization: The JobManager (running within a container on a YARN NodeManager) initializes the Flink job.
2. Task Deployment: The JobManager assigns tasks to different TaskManagers (also running in containers on YARN NodeManagers).
3. Data Flow: Data is processed as it flows through the tasks of the Flink job.
4. State Management: TaskManagers manage their state, periodically saving it to checkpoints for fault tolerance.
5. Fault Recovery: If a TaskManager fails, the JobManager uses the latest checkpoints to restore the state and restart the task on a different TaskManager.

Experiment 1

I had a virtual Ubuntu already installed on my pc. So, I started from the step 1.2 which is downloading apache flink.

```
vboxuser@BM1: ~  
vboxuser@BM1:~$ wget https://dlcdn.apache.org/flink/flink-1.19.1/flink-1.19.1-bin-scala_2.12.tgz  
--2024-10-13 13:28:51-- https://dlcdn.apache.org/flink/flink-1.19.1/flink-1.19.1-bin-scala_2.12.tgz  
Resolving dlcdn.apache.org (dlcdn.apache.org)... 151.101.2.132, 2a04:4e42::644  
Connecting to dlcdn.apache.org (dlcdn.apache.org)|151.101.2.132|:443... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 482499797 (460M) [application/x-gzip]  
Saving to: 'flink-1.19.1-bin-scala_2.12.tgz'  
  
flink-1.19.1-bin-sc 100%[=====] 460,15M 6,96MB/s in 60s  
  
2024-10-13 13:29:52 (7,62 MB/s) - 'flink-1.19.1-bin-scala_2.12.tgz' saved [482499797/482499797]  
  
vboxuser@BM1:~$
```

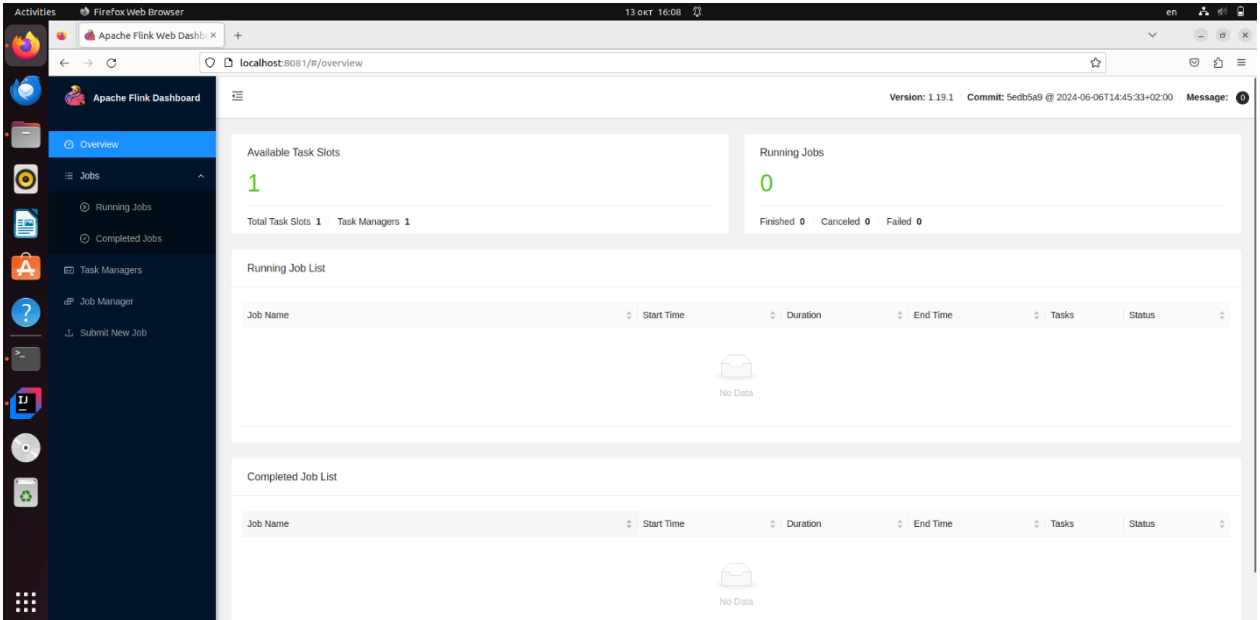
```
vboxuser@BM1:/$ cd /home/vboxuser/Documents  
vboxuser@BM1:~/Documents$ tar -xvzf flink-1.19.1-bin-scala_2.12.tgz  
flink-1.19.1/  
flink-1.19.1/lib/  
flink-1.19.1/lib/log4j-1.2-api-2.17.1.jar  
flink-1.19.1/lib/flink-table-api-java-uber-1.19.1.jar  
flink-1.19.1/lib/flink-csv-1.19.1.jar  
flink-1.19.1/lib/flink-scala_2.12-1.19.1.jar  
flink-1.19.1/lib/flink-connector-files-1.19.1.jar  
flink-1.19.1/lib/log4j-core-2.17.1.jar  
flink-1.19.1/lib/log4j-slf4j-impl-2.17.1.jar
```

Configuring Java

```
vboxuser@BM1:~/Documents/flink-1.19.1$ sudo apt install openjdk-17-jre-headless
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ca-certificates-java java-common
Suggested packages:
  default-jre fonts-dejavu-extra fonts-ipafont-gothic fonts-ipafont-mincho
  fonts-wqy-microhei | fonts-wqy-zenhei
The following NEW packages will be installed:
  ca-certificates-java java-common openjdk-17-jre-headless
0 to upgrade, 3 to newly install, 0 to remove and 77 not to upgrade.
Need to get 48,2 MB of archives.
```

Trying to start the cluster, success; then stopping cluster for now.

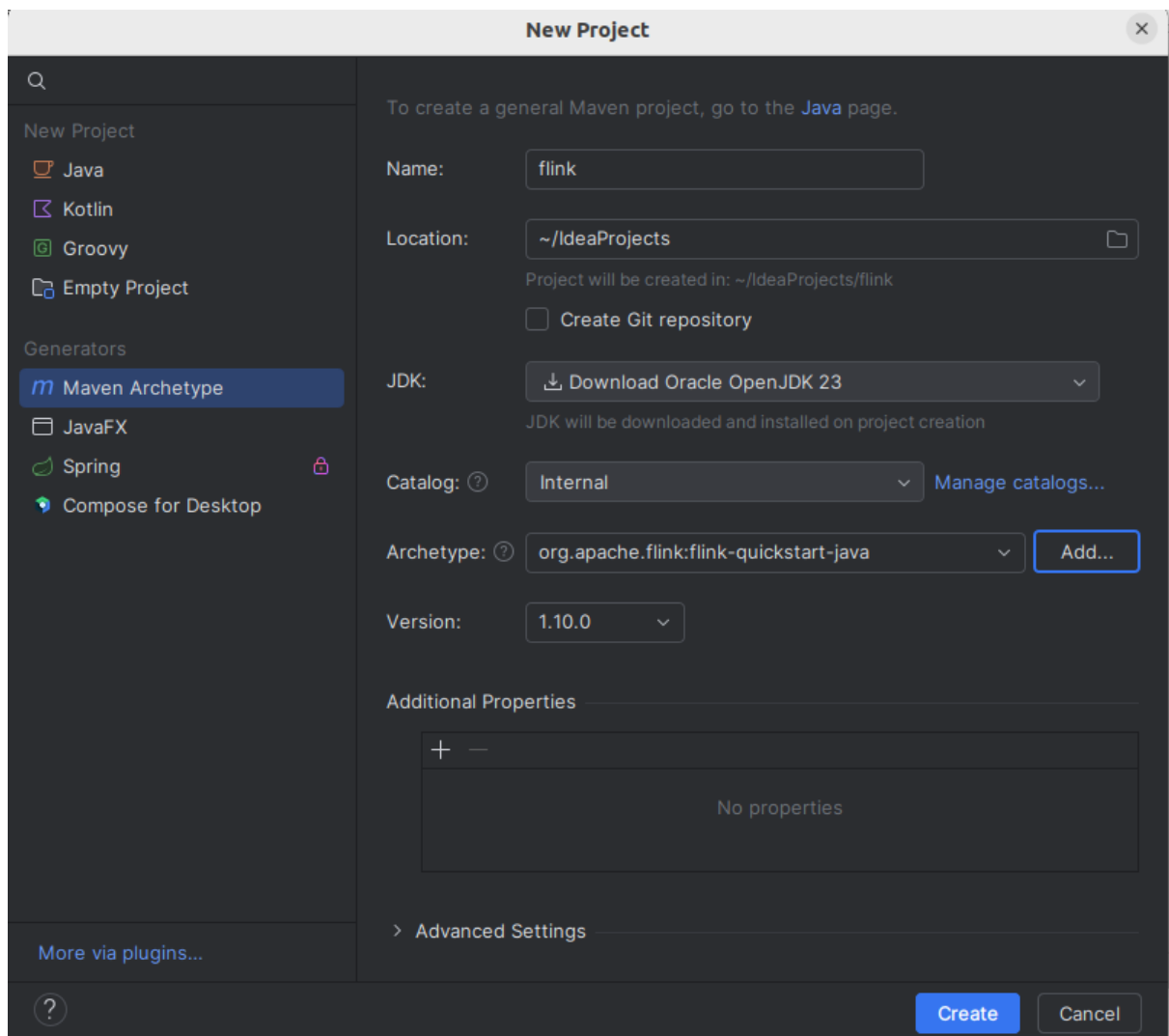
```
vboxuser@BM1:~/Documents/flink-1.19.1$ ./bin/start-cluster.sh
Starting cluster.
Starting standalone session daemon on host BM1.
Starting taskexecutor daemon on host BM1.
```



The screenshot shows the Apache Flink Dashboard web interface. The browser address bar indicates the URL is `localhost:8081/#/overview`. The dashboard header shows the version as 1.19.1 and the commit as 5ed5a9. The main content area displays the following information:

- Available Task Slots:** 1
- Running Jobs:** 0
- Finished:** 0, **Canceled:** 0, **Failed:** 0
- Running Job List:** A table with columns for Job Name, Start Time, Duration, End Time, Tasks, and Status. It currently shows "No Data".
- Completed Job List:** A table with columns for Job Name, Start Time, Duration, End Time, Tasks, and Status. It also shows "No Data".

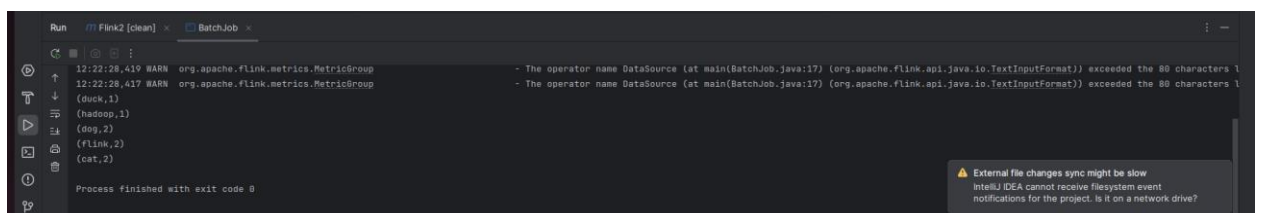
Creating a new project in IntelliJ IDEA with Maven Archetype and adding a new archetype for Apache Flink.



Doing the mvn clean install operation

```
vboxuser@BM1:~/IdeaProjects/flink$ mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.example:flink >-----
[INFO] Building Flink Quickstart Job 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-p
```

Trying to run and compile code BatchJob. Result is showed below.



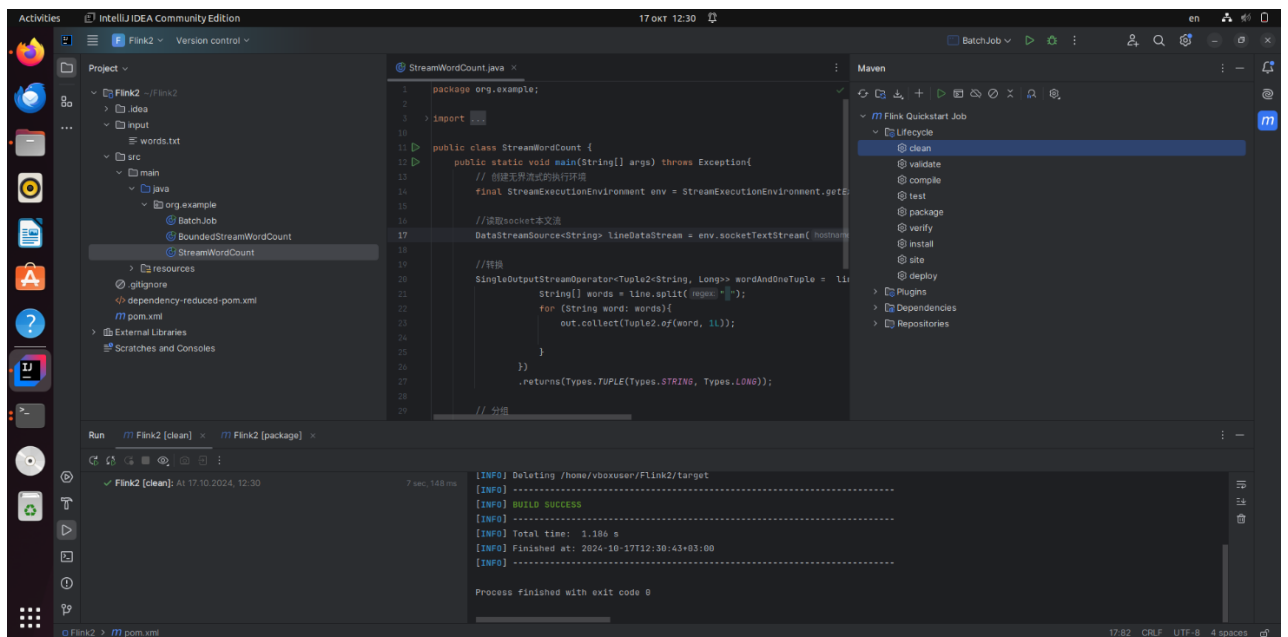
Installing netcat-openbsd

```
vboxuser@BM1: ~  
Reading state information... Done  
92 packages can be upgraded. Run 'apt list --upgradable' to see them.  
vboxuser@BM1:~$ sudo apt install netcat-openbsd  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
netcat-openbsd is already the newest version (1.218-4ubuntu1).  
netcat-openbsd set to manually installed.  
0 to upgrade, 0 to newly install, 0 to remove and 92 not to upgrade.  
vboxuser@BM1:~$ nc -h  
OpenBSD netcat (Debian patchlevel 1.218-4ubuntu1)  
usage: nc [-46CDdFhklNnrStUuvZz] [-I length] [-i interval] [-M ttl]  
        [-m minttl] [-O length] [-P proxy_username] [-p source_port]  
        [-q seconds] [-s sourceaddr] [-T keyword] [-V rtable] [-W recvlimit]  
        [-w timeout] [-X proxy_protocol] [-x proxy_address[:port]]  
        [destination] [port]  
Command Summary:  
-4          Use IPv4  
-6          Use IPv6  
-b          Allow broadcast  
-C          Send CRLF as line-ending  
-D          Enable the debug socket option  
-d          Detach from stdin  
-F          Pass socket fd
```

Meanwhile starting cluster.

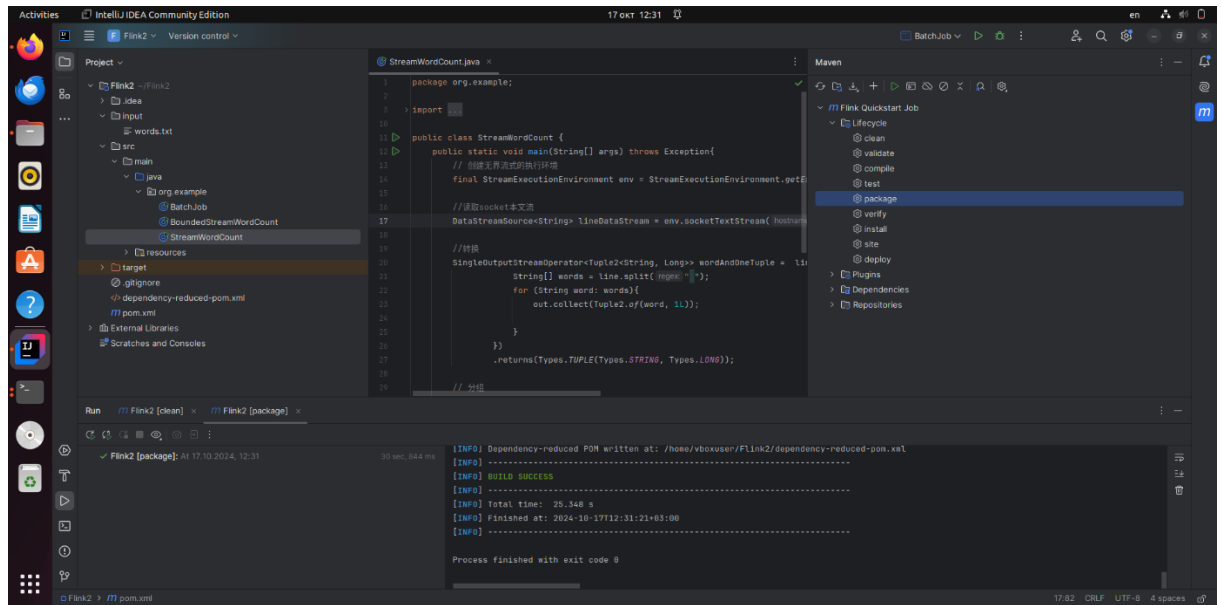
```
vboxuser@BM1: ~/Documents/flink-1.19.1  
vboxuser@BM1:~/Documents/flink-1.19.1$ ./bin/start-cluster.sh  
Starting cluster.  
Starting standalone session daemon on host BM1.  
Starting taskexecutor daemon on host BM1.  
vboxuser@BM1:~/Documents/flink-1.19.1$
```

Doing clean and package for the StreamWordCount.

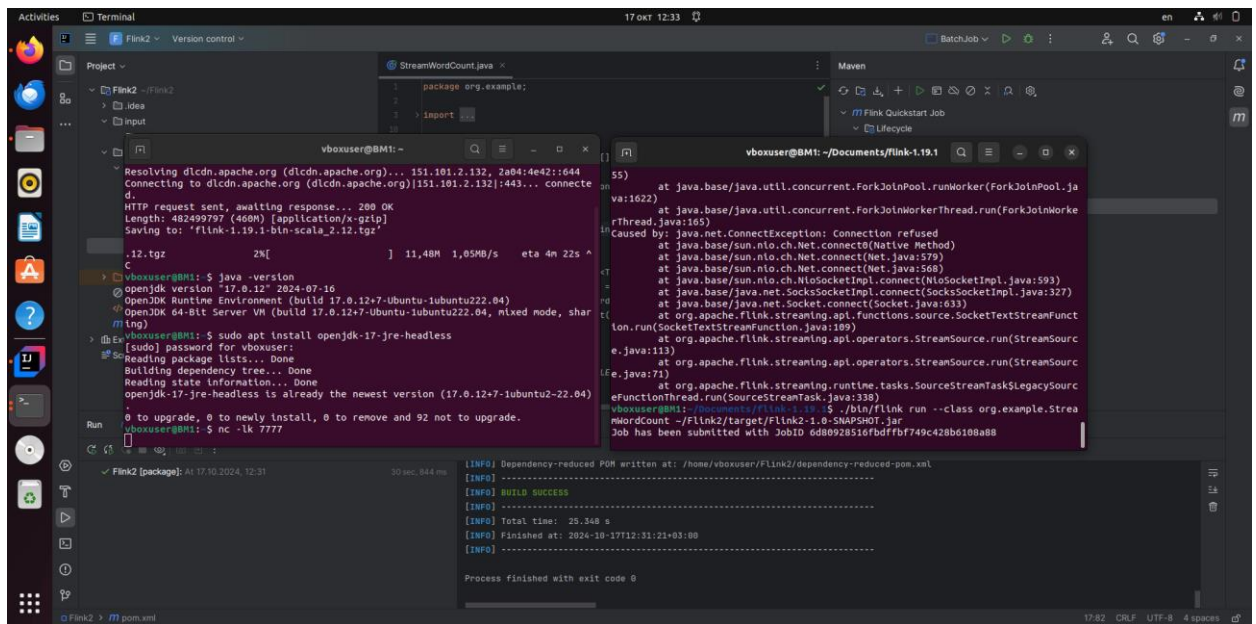


```
StreamWordCount.java  
1 package org.example;  
2  
3 import java.util.*;  
4  
5  
6  
7  
8  
9  
10  
11 public class StreamWordCount {  
12     public static void main(String[] args) throws Exception {  
13         // 创建无界流式的执行环境  
14         *final StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();  
15  
16         // 读取socket文本流  
17         DataStreamSource<String> lineDataStream = env.socketTextStream("localhost", 7777);  
18  
19         // 转换  
20         SingleOutputStreamOperator<Tuple2<String, Long>> wordAndOneTuple = lineDataStream.map(new MapFunction<String, Tuple2<String, Long>>() {  
21             String[] words = line.split(" ");  
22             for (String word : words) {  
23                 out.collect(Tuple2.of(word, 1));  
24             }  
25         });  
26  
27         .returns(Types.TUPLE(Types.STRING, Types.LONG));  
28  
29         // 分组
```

```
Maven  
Lifecycle  
clean  
validate  
compile  
test  
package  
verify  
install  
site  
deploy  
Plugins  
Dependencies  
Repositories  
BatchJob  
Flink Quickstart Job  
Lifecycle  
clean  
validate  
compile  
test  
package  
verify  
install  
site  
deploy  
Plugins  
Dependencies  
Repositories  
Run  
Flink2 [clean] x Flink2 [package]  
Flink2 [clean]: At 17:10 2024, 12:30  
7 sec, 148 ms  
[INFO] Deleting /home/vboxuser/Flink2/target  
[INFO] -----  
[INFO] BUILD SUCCESS  
[INFO] Total time: 1.186 s  
[INFO] Finished at: 2024-10-17T12:30:43+08:00  
[INFO] -----  
Process finished with exit code 0
```

Doing the nc command and running the jar package in other terminal.



Opening localhost:8081. Opening the Flink Streaming Job. Now in stdout we can see the words I put in the terminal.

Activities Firefox Web Browser 17 OCT 12:33 en

Apache Flink Web Dashboard

localhost:8081/#/overview

Version: 1.19.1 Commit: 5ed5e9 @ 2024-06-06T14:45:33+02:00 Message: 0

Available Task Slots: 0

Total Task Slots 1 Task Managers 1

Running Jobs: 1

Finished 0 Canceled 0 Failed 2

Running Job List

Job Name	Start Time	Duration	End Time	Tasks	Status
Flink Streaming Job	2024-10-17 12:33:27.559	21s 74ms	-	2 2	RUNNING

Completed Job List

Job Name	Start Time	Duration	End Time	Tasks	Status
Flink Streaming Job	2024-10-17 12:29:43.107	3s 952ms	2024-10-17 12:29:47.059	2 1 1	FAILED
Flink Streaming Job	2024-10-17 12:31:49.573	1s 651ms	2024-10-17 12:31:51.224	2 1 1	FAILED

Activities Firefox Web Browser 17 OCT 12:36 en

Apache Flink Web Dashboard

localhost:8081/#/job/running/ed80928516bdfbf749c428b6108a88/overview/90bea6de1c231edf33913ecd54406c1/taskmanagers

Cancel Job

Flink Streaming Job

Job ID	6d80928516bdfbf749c428b6108a88	Job State	RUNNING 2	Actions	Job Manager Log
Start Time	2024-10-17 12:33:27.559	Duration	2m 49s 356ms		

Overview Exceptions TimeLine Checkpoints Configuration

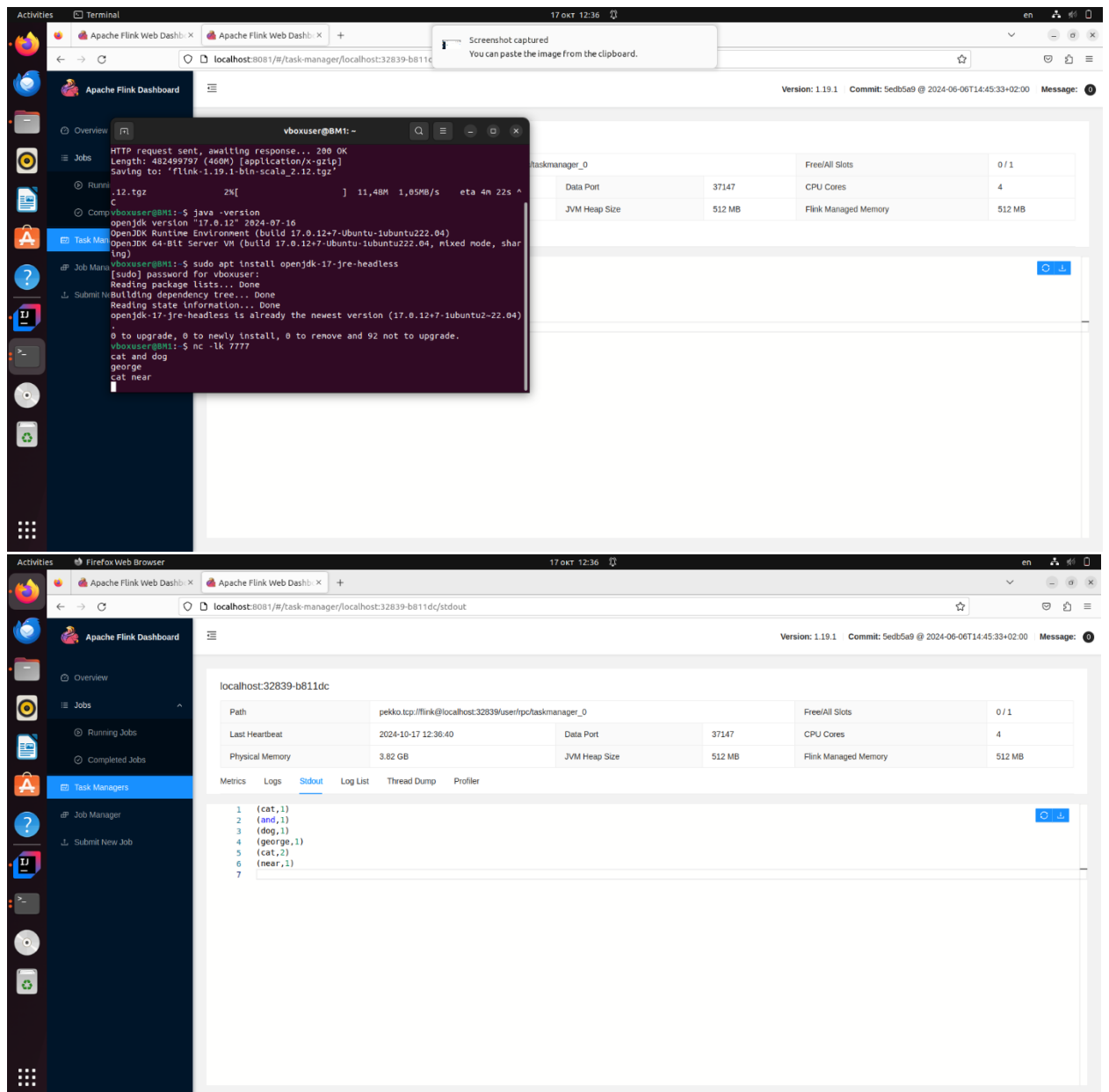
Detail SubTasks TaskManagers Watermarks Accumulators BackPressure Metrics FlameGraph

Endpoint	Bytes received	Records received	Bytes sent	Records sent	Start Time	Status	More
localhost:37147	75 B	4	0 B	0	2024-10-17 12:33:27.559	RUNNING	

Job Graph

```
graph LR; A[Source: Socket Stream -> Flat Map] -- HASH --> B[Keyed Aggregation -> Sink: Print to Std. Out]; B --> C[Parallelism: 1]; C --> D[Backpressured (max): 0% Busy (max): 0%];
```

Name	ID	Records Received	Bytes Sent	Records Sent	Parallelism	Start Time	Duration	End Time	Tasks
Source: Socket Stream -> Flat Map	1	0	0 B	4	1	2024-10-17 12:33:27.997	2m 48s 918ms	-	1
Keyed Aggregation -> Sink: Print to Std. Out	2	4	0 B	0	1	2024-10-17 12:33:28.005	2m 48s 910ms	-	1



Here you can see that I put in the terminal:

cat and dog
george
cat near

And in stdout we can see:

(cat,1)
(and,1)
(dog,1)
(george,1)
(cat,2)
(near,1)